Stellar Classification (SDSS)

Using Multi-Layer Perceptron

Marco Siciliano
Fabio Desio Suarez
Luca Nathansohn

Workflow & Methodology

Tools & Frameworks

- **Environment:** Jupyter Notebook, `scikit-learn`.
- **Generative AI:** Gemini Pro, Claude.

Collaborative Strategy

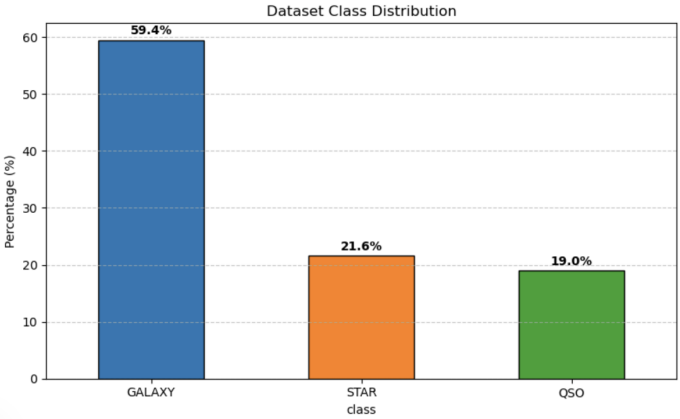
1. **Independent Exploration**
2. **Prompt Engineering**
3. **Peer Review**
4. **Synthesis**

The Physical System: SDSS Stellar Dataset

- **Objective:** Classify celestial objects into 3 classes (Stars, Galaxies, Quasars) based on photometric data.

Dataset size: 100 000

[Source: Kaggle]



The Physical System: SDSS Stellar Dataset

- **Physical Features:** The dataset provides 5 photometric filters (u, g, r, i, z) and the spectroscopic *redshift*.

u	g	r	i	z	class	redshift
23.879	22.275	20.395	19.166	18.794	GALAXY	0.63479
24.778	22.832	22.584	21.168	21.614	GALAXY	0.77914
25.263	22.664	20.610	19.349	18.948	GALAXY	0.64419
22.137	23.777	21.612	20.505	19.250	GALAXY	0.93235
19.437	17.580	16.497	15.977	15.545	GALAXY	0.11612

The Physical System: SDSS Stellar Dataset

- **Data Integrity Challenges:**
  - *Spurious Correlations:* The dataset includes telescope metadata (e.g., `run_ID`).
  - *Hardware Anomalies:* Sensor errors with non-physical conventions (e.g., -9999).

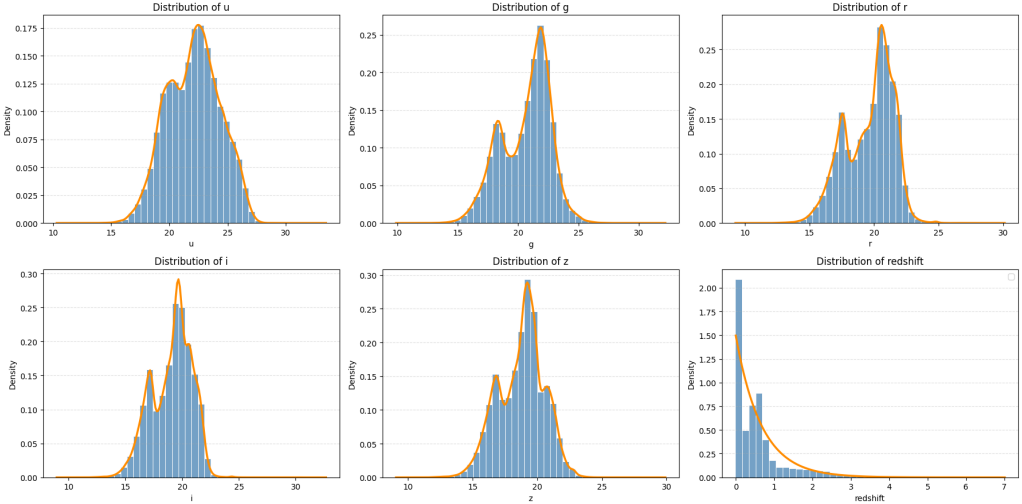
```python
columns_to_load = ['class', 'u', 'g', 'r', 'i', 'z', 'redshift']
df = pd.read_csv(file_path, usecols=columns_to_load)

for col in ['u', 'g', 'r', 'i', 'z']:
    df = df[df[col] > -30]

```

By cutting values below -30, we eliminate artificial spikes (-9999).
This is a physical boundary: the Sun has an apparent magnitude of ~ -26.7.

Data Preparation: Splitting and Scaling

1. **The 3-Way Split** (60%, 20%, 20%)
   - **Train:** Dynamic update of parameters W and b .
   - **Validation:** Hyperparameter selection (layers, neurons, learing rate) without biasing the model.
   - **Test:** Unbiased final evaluation.
2. **Feature Rescaling**
   - We selected `RobustScaler` (Median and IQR) to mathematically shield the metric from natural outliers (e.g., ultra-luminous quasars).

Data Preparation: Splitting and Scaling



The Neural Network: Multilayer Perceptron (MLP)

Given the hyperparameters L (layers) and $\{n_i\}_{i=0}^{L}$ (neurons), the MLP is a composition of affine functions and non-linear activations σ.

$$\mathbb{R}^{n_0} \xrightarrow[\sigma]{W^{(1)}} \mathbb{R}^{n_1} \xrightarrow[\sigma]{W^{(2)}} \mathbb{R}^{n_2}$$

The complete forward pass is computed as:

$$f(x) = \sigma\left(W^{(L)}\sigma\left(W^{(L-1)}\ldots\sigma\left(W^{(1)}x + b^{(1)}\right)\cdots + b^{(L-1)}\right) + b^{(L)}\right) \tag{1}$$

Gradient Descent & SGD

For a cost function E, Gradient Descent operates via:

$$x_{k+1} = x_k - \eta \nabla E(x_k) \tag{2}$$

Stochastic Gradient Descent (SGD): Computing the exact gradient on the entire dataset is computationally expensive. SGD approximates the gradient using a random subset of data (*mini-batch*) B:

$$x_{k+1} = x_k - \eta \nabla E_B(x_k) \quad \text{where} \quad E_B = \frac{1}{|B|} \sum_{i \in B} E_i(x_k) \tag{3}$$

This stochastic noise helps the algorithm escape local minima in non-convex spaces.

Implementation

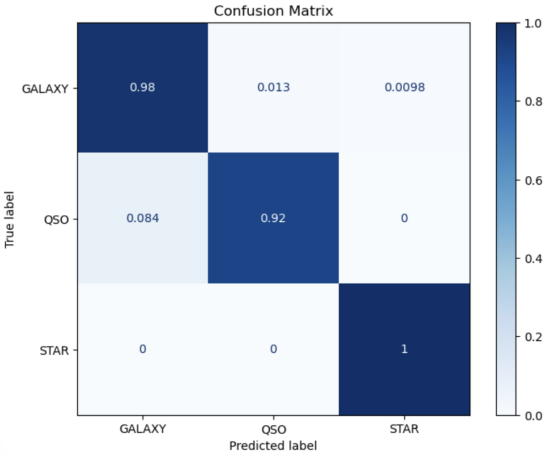
```python
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.25, random_state=42, stratify=y_temp
)

scaler = RobustScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)

learning_rates = [0.2, 0.1, 0.01]
hidden_layer_configs = [(1,), (2,), (4,)]
```

Implementation

```python
for lr in learning_rates:
    for hl in hidden_layer_configs:
        mlp = MLPClassifier(
            hidden_layer_sizes=hl,
            activation='logistic',
            solver='sgd',
            batch_size=128,
            learning_rate_init=lr,
            max_iter=500,
            random_state=42
        )
        mlp.fit(X_train_scaled, y_train)
        accuracy_val = mlp.score(X_val_scaled, y_val)

        if accuracy_val > best_accuracy_val:
            best_accuracy_val = accuracy_val
            best_params = {'learning_rate_init': lr, 'hidden_layer_sizes': hl}
            best_model = mlp

```

Initial Results

- A minimalist network (1 layer, 4 neurons, $\eta = 0.2$) achieved $\sim 97\%$ accuracy on the entire dataset.
- The confusion matrix confirms there is **no overfitting** on the Test Set.

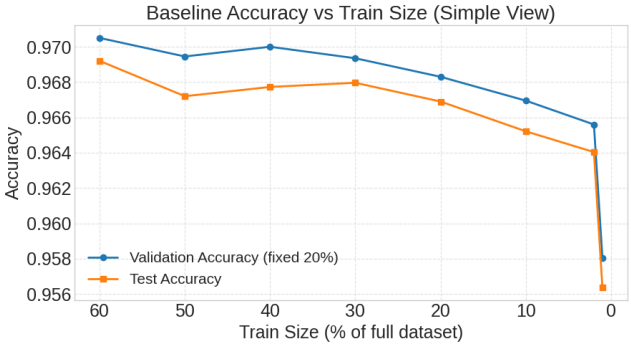


How robust is our architecture?

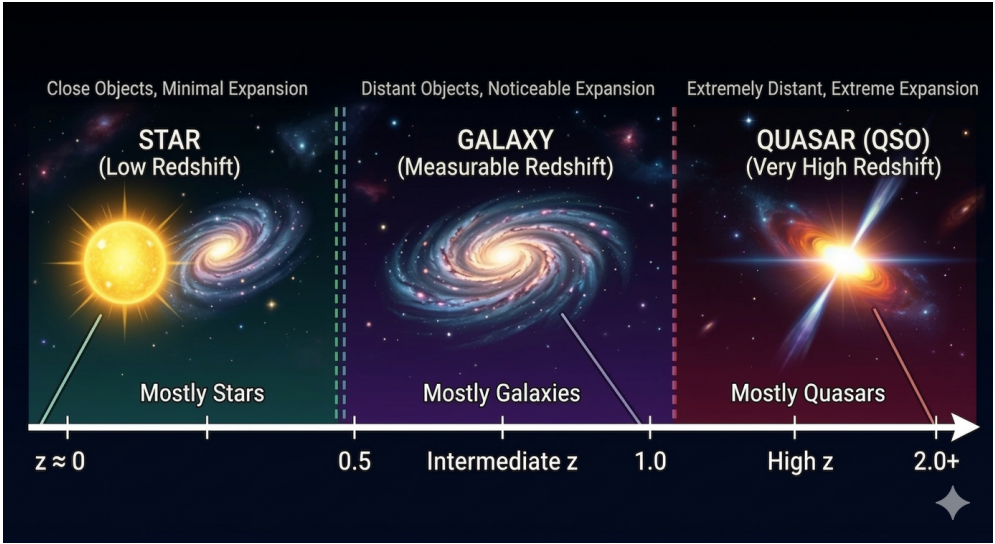
How much information does it actually need to map this space?

Training using 1% Dataset

- *Result:* Accuracy remained almost unaffected.
- *Hypothesis:* The problem geometry must be unusually simple to maintain such predictive power with so little data.

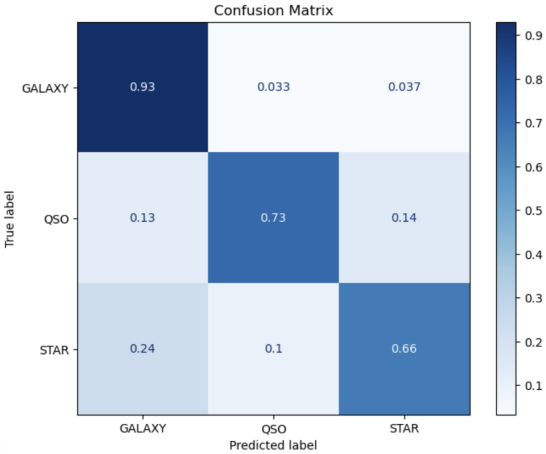


Linear Separability by Redshift



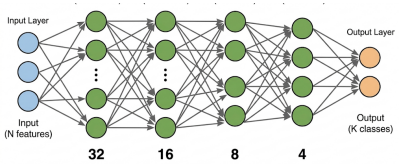
Downgrade Without Redshift

- Removing the redshift forces the simple network to rely purely on photometry.
- Without the kinematic separation, the minimalist model fails to handle the inherent degeneracy of the features ($\sim$ 83% accuracy).

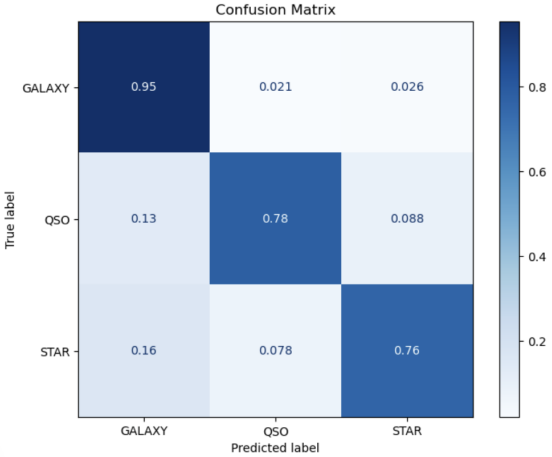


Classification Without Redshift

- We are forced to use a much more complex network.



- Despite the added complexity, accuracy sets around $\sim 88\%$.



We run into an intrinsic photometric degeneracy that the network cannot overcome.

AI as a Collaborative Tool

"Generate clean, well-commented Python code for a Jupyter Notebook using Scikit-learn. The goal is to build the simplest possible neural network to classify stellar objects as galaxies, stars, or quasars based on photometric and spectroscopic features. The target variable is class. Use only the following 6 input features: the 5 photometric bands u, g, r, i, z and the redshift. Split the dataset: train 60%, validation 20%, test 20%. The model must be an MLPClassifier from Scikit-learn. Keep the architecture as simple as possible. Use logistic sigmoid as the activation function and SGD. The code use a grid search to compare three architectures with 1, 2, and 4 hidden layers. Add a confusion matrix to see the normalized recall for the best model. Keep the code minimal and skeleton-level with comments."

- **Cautious Trust:** The AI acts as an interactive sounding board. Code and conceptual explanations were systematically verified against physical principles.

Thank You

Any questions?

Backup I: Base Model (with Redshift)

Optimization Results

- Best Parameters: `learning_rate_init`: 0.2, `hidden_layer_sizes`: (4,)
- Train Accuracy: 0.9683
- Validation Accuracy: 0.9692
- Test Accuracy (unseen data): **0.9705**

Class	Precision	Recall	F1-score	Support
GALAXY	0.97	0.98	0.98	11889
QSO	0.96	0.92	0.94	3792
STAR	0.97	1.00	0.99	4319
Accuracy			*0.97*	*20000*
Macro Avg	0.97	0.96	0.97	20000
Weighted Avg	0.97	0.97	0.97	20000

Backup II: 1% Training Data (with Redshift)

Optimization Results

- Validation Accuracy: 0.96045
- Test Accuracy (unseen data): **0.9551**

Class	Precision	Recall	F1-score	Support
GALAXY	0.96	0.96	0.96	46962
QSO	0.95	0.88	0.92	14980
STAR	0.95	0.99	0.97	17059
Accuracy			*0.96*	*79001*
Macro Avg	0.95	0.95	0.95	79001
Weighted Avg	0.96	0.96	0.95	79001

Backup III: Simple Model (Without Redshift)

Optimization Results

- Best Parameters: `learning_rate_init`: 0.2, `hidden_layer_sizes`: (4,)
- Validation Accuracy: 0.8367
- Test Accuracy (unseen data): **0.8336**

Class	Precision	Recall	F1-score	Support
GALAXY	0.88	0.93	0.90	11889
QSO	0.77	0.73	0.75	3792
STAR	0.74	0.66	0.70	4319
Accuracy			*0.83*	*20000*
Macro Avg	0.80	0.77	0.78	20000
Weighted Avg	0.83	0.83	0.83	20000

Backup IV: Complex Model (Without Redshift)

Optimization Results

- Best Parameters: `learning_rate_init`: 0.2, `hidden_layer_sizes`: (32, 16, 8, 4)
- Validation Accuracy: 0.8830
- Test Accuracy (unseen data): **0.8790**

Class	Precision	Recall	F1-score	Support
GALAXY	0.90	0.95	0.93	11889
QSO	0.84	0.78	0.81	3792
STAR	0.84	0.76	0.80	4319
Accuracy			*0.88*	*20000*
Macro Avg	0.86	0.83	0.84	20000
Weighted Avg	0.88	0.88	0.88	20000